



《程序设计课程设计》

课设报告

班 级： 软件 2429

题 目： 校园即时通信与文件传输系统

组 长： 李坤灿 指导老师： 熊小樵

组 员： 侯皓骞

董 鑫

软件学院

二〇二六年

项目答辩记录表

班级：软件2429	项目成员姓名：李坤灿 侯皓骞 董鑫
项目题目：	校园即时通信与文件传输系统
项目描述：描述项目功能、实现方法、待解决的问题等 一、项目概述 校园即时通信与文件传输系统是一个基于 C/S 架构的局域网即时通信软件，为师生提供实时文字聊天和文件传输服务。服务端采用 Python 多线程 TCP Socket，客户端使用 tkinter GUI，数据持久化使用 SQLite。 【技术栈】 Python 3.12 + TCP Socket + tkinter + SQLite + JSON 协议 + Base64 编码 + PyInstaller 1.1 项目目标 • 多用户同时在线的即时通信 • 点对点私聊和公共群聊 • 文件传输（含进度显示） • 聊天记录持久化与历史查询 • 在线状态实时同步 • 异常断线自动检测与清理 二、项目功能 2.1 用户认证 • 注册：用户名唯一性检查，密码 SHA256 哈希存储 • 登录：密码验证，防止重复登录 • 登出：主动退出或异常断线自动清理 2.2 在线状态管理 • 登录时返回在线用户列表	

- 上线/下线广播通知所有用户
- 在线列表实时更新

2.3 点对点私聊

- 选择在线用户发起私聊，独立聊天窗口
- 消息通过 MessageRouter 精确路由
- 消息持久化，支持历史查询

2.4 群聊

- 所有在线用户自动加入公共聊天室
- 消息广播给所有在线用户（排除发送者）
- 群聊消息持久化

2.5 文件传输

- 发送方发起请求 → 接收方确认 → 分块传输 → 完成通知
- Base64 分块编码（每块 64KB）嵌入 JSON
- 实时进度显示，文件保存到 received_files/
- 同名文件自动加序号

2.6 心跳保活

- 客户端每 30 秒发送心跳包
- 服务端 60 秒无响应自动清理死连接
- 断线后广播下线通知

三、系统架构

3.1 整体架构

采用 C/S 架构。服务端集中管理连接、认证、路由和持久化。客户端负责 UI 展示。两者通过 TCP Socket 长连接通信，JSON 格式封装消息，换行符分隔。

3.2 服务端核心类

- ChatServer：监听端口 8888，接受连接，启动处理线程

- ClientHandler: 每连接一线程, 收发消息、路由、响应
- MessageRouter: 策略模式, {MsgType: handler} 映射分发
- ClientManager: 线程安全用户管理, Lock 保护字典

3.3 客户端架构

- ChatClient: Socket 连接、消息收发、心跳、回调分发
- 监听线程: 持续接收消息, 按类型触发回调
- 心跳线程: 每 30 秒发送 HEARTBEAT
- UI 层: tkinter, 通信层与 UI 层通过回调解耦

3.4 数据库

- user 表: username (主键)、password (SHA256)、nickname、status
- private_message 表: sender、receiver、content、timestamp
- group_message 表: sender、group_id、content、timestamp

3.5 通信协议

JSON 统一格式, 17 种消息类型:

- 认证类: REGISTER、LOGIN、LOGOUT
- 状态类: ONLINE_LIST、USER_ONLINE、USER_OFFLINE
- 聊天类: PRIVATE_MSG、GROUP_MSG
- 文件类: FILE_REQUEST、FILE_RESPONSE、FILE_DATA、FILE_COMPLETE
- 历史类: HISTORY_REQUEST、HISTORY_RESPONSE
- 系统类: HEARTBEAT、ERROR、SUCCESS、QUIT、PROGRESS

四、实现方法

4.1 消息路由

MessageRouter 采用策略模式, 消息类型映射到处理函数。新增消息类型只需添加处理函数和一行映射, 符合开闭原则。

4.2 线程安全

ClientManager 使用 Lock 保护字典, broadcast() 用快照遍历避免并发修改。数据库

操作加锁防止 SQLite 写入冲突。

4.3 文件传输

5 步握手：请求 → 确认 → 分块传输（独立线程） → 完成通知 → 保存文件。

4.4 心跳保活

客户端 30 秒心跳，服务端 60 秒超时。超时后清理用户、更新数据库、广播下线。允许丢失一个心跳包。

4.5 TCP 粘包处理

以换行符分隔消息。发送端追加 \n，接收端维护 buffer 按 \n 分割解析。

4.6 GUI 与通信分离

ChatClient 定义 16 种回调事件，UI 层通过 set_callback() 注册。通信核心不依赖 GUI 框架，便于测试。

五、待解决的问题与改进方向

5.1 安全性

【密码存储】 SHA256 无加盐，易受彩虹表攻击。改进：使用 bcrypt/Argon2。

【通信加密】 明文 JSON 传输。改进：引入 TLS/SSL 加密。

【输入验证】 缺乏长度限制和特殊字符过滤。改进：添加输入校验。

5.2 性能

【并发模型】 每连接一线程，高并发时线程开销大。改进：改用 asyncio/epoll。

【文件传输】 Base64 增加 33% 数据量。改进：二进制直传 + 长度前缀。

【数据库】 SQLite 文件级锁限制并发。改进：迁移 MySQL/PostgreSQL 或 WAL 模式。

5.3 功能完善

【离线消息】 用户不在线时消息丢失。改进：服务端缓存，上线后推送。

【断点续传】 协议已预留 offset 字段但未实现。改进：记录 offset 从断点继续。

【好友系统】 无好友分组。改进：添加好友关系表。

【消息扩展】仅支持文本和文件。改进：支持图片、表情、语音等。

5.4 工程化

【日志】当前用 `print()`。改进：引入 `logging` 模块。

【配置】端口等硬编码。改进：YAML/JSON 配置文件。

【部署】手动启动。改进：Docker 一键部署。

六、项目总结

本项目实现了用户认证、私聊、群聊、文件传输、历史记录、在线状态管理六大核心功能。采用 C/S 架构，Python 多线程 TCP Socket + JSON 协议 + SQLite 持久化。编写 50 个测试用例，100% 通过。

系统虽在安全性、性能、功能完善度方面仍有不足，但作为课程设计，已充分展示软件工程方法和网络编程核心技术。上述待解决问题为后续改进指明了方向。

答辩中提出的主要问题及回答的简要情况：

李坤灿：

Q: 客户主动断开链接后，服务器会怎么做？

A: 服务器首先会通过TCP的四次挥手，感知到连接断开，然后从在线列表移除、数据库设成offline，最后广播下线通知

侯皓骞：

Q: 项目中所使用的是是什么协议？ A: TCP

Q: Tcp的三次握手是什么Tcp？除了三次握手还有什么 A: 客户端向服务端发送 SYN 包，请求建立连接；服务端收到后，回复 SYN+ACK 包，表示同意；客户端再发送 ACK 包，连接正式建立。还有四次挥手

Q: 四次挥手的内容是什么？

项目评分
(满分100分)

A: 四次挥手是 TCP 断开连接的过程：客户端发送 FIN 包；服务端收到后回复 ACK；服务端处理完剩余数据后，也发送 FIN 包；客户端回复 ACK。

董鑫：

Q: 如何判断文件传输成功：

A: 上传成功 = 服务器给你发了"收到"的回执
你传完文件，服务器必须返回一个文件ID或成功标记。

没收到这个回执 = 没传成功，必须重传。

文件没损坏 = 比一下"指纹"

发之前算一个MD5值（相当于文件的身份证号），带着一起上传。

服务器收完后也算一遍，对得上才存盘，对不上就丢弃并让你重传。

对方真正拿到了 = 对方发回"下载完成"的确认

对方下载完成后，也算一遍MD5，对得上才发回执说"我收到了"。

你这边看到双勾或"已下载"标记，才算整个流程结束。

指导教师签名：

年 月 日

姓名	角色	任务详情	开发小组内成员互评	评分
李坤灿	组长	服务端核心开发（多线程并发、用户管理、消息路由） 聊天记录持久化模块 通信协议定义与文档 需求分析文档主导 系统设计文档主导 最终项目文档整合 Git 仓库管理与项目进度统筹	侯皓骞：工作认真负责，客户端框架搭建及时，测试用例设计全面，积极配合联调 董鑫：群聊和文件传输模块完成度高，文档编写规范，协作沟通顺畅	100
2429				
侯皓骞	组员	客户端框架（Socket 连接、登录、在线列表） 点对点私聊模块系统测试负责人 测试计划制定与测试用例设计 多客户端联调测试与回归测试测试报告撰写	李坤灿：服务端架构设计清晰，协议定义规范，项目进度把控到位 董鑫：模块开发效率高，代码质量良好，文档协助及时	100
2429				
董鑫	组员	群聊/公共聊天室模块 文件传输模块（选择、发送、接收、进度显示） README.md 与运行说明 文档格式排版协助	李坤灿：技术能力强，问题解决及时，文档整合专业 侯皓骞：测试工作细致，bug 反馈准确，联调配合积极	100
2429				

任 务 书

项目名称	校园即时通信与文件传输系统	设计周数	2
设计 任务 主要 设计 参数	小组规模：3 人 系统架构：客户端/服务器（C/S）架构 编程语言：Python 3.10+ 通信协议：TCP Socket + JSON 消息封装 数据库：SQLite 3 GUI 框架：tkinter（Python 标准库） 开发周期：10 天（2026 年6 月 15 日-6 月25 日）		
设计内容 设计要求	本题目采用 C/S 架构，要求完成以下功能： 1) 采用 C/S 架构，至少包含一个服务端和一个客户端程序。 2) 支持用户注册和登录功能，密码采用 SHA256 哈希存储。 3) 支持在线用户列表实时显示。 4) 支持点对点文本聊天（私聊），消息以气泡形式展示。 5) 支持群聊/公共聊天室功能，消息广播给所有在线用户。 6) 支持文件在线传输，包含传输进度显示。 7) 支持聊天历史记录查询。 8) 具备心跳保活和断线重连机制。 9) 服务端支持多客户端并发连接，单个客户端异常不影响服务端运行。		

主要参考资料	谢希仁.《计算机网络》. 电子工业出版社。 Python Socket 编程官方文档： https://docs.python.org/3/library/socket.html SQLite 官方文档: https://docs.python.org/3/library/sqlite3.html tkinter 官方文档： https://docs.python.org/3/library/tkinter.html
学生提交 归档文件	(1) 项目组文件夹： ①程序源代码 ②需求分析文档 ③系统设计文档 ④最终项目文档 ⑤项目运行说明（README.md） (2) 个人文件夹： ①个人开发日志

注：

1. 课程设计完成后，学生提交的归档文件应按照：封面—答辩记录表和项目分工表—任务书—正文—源代码—个人开发日志上交
2. 可根据实际内容需要续表，但应保持原格式不变。

指导教师签名：

日期：

目 录

一、需求分析.....	1
1.1 问题定义.....	1
1.2 可行性研究.....	1
1.3 功能需求.....	1
1.4 非功能需求.....	2
1.5 用例分析.....	2
1.6 数据流图.....	3
1.7 E-R 图与数据字典.....	3
1.8 验收标准.....	3
二、概要设计.....	4
2.1 系统架构设计.....	4
2.2 模块划分.....	4
2.3 技术选型.....	5
三、详细设计.....	5
3.1 通信协议设计.....	5
3.2 数据库设计.....	6
3.3 核心类设计.....	6
3.4 核心流程设计.....	7
3.5 界面设计.....	7
四、系统实现.....	8
4.1 开发环境与工具.....	8
4.2 服务端实现.....	8
4.3 客户端实现.....	9
4.4 关键功能实现说明.....	10
4.5 实际实现与设计调整.....	10
五、系统测试.....	11
5.1 测试策略与环境.....	11
5.2 功能测试.....	11
5.3 异常测试.....	12

5.4 并发测试.....	12
5.5 测试结果汇总.....	12
核心源代码.....	13
AI 辅助开发说明.....	14

一、需求分析

1.1 问题定义

1.1.1 背景

在校园环境中，学生之间、师生之间需要进行频繁的即时沟通。现有的即时通信工具（如微信、QQ）虽然功能完善，但存在以下不足：依赖外部网络，需要连接互联网，在校园局域网或网络受限环境下无法使用；功能冗余，通用 IM 软件功能过于庞杂，与教学场景无关的广告、娱乐内容分散注意力；文件传输受限，部分工具对文件大小、格式有限制，且传输依赖云端中转，速度和安全不可控；缺乏教学定制，无法针对课程设计需求进行功能裁剪和技术分析。

本项目旨在设计并实现一个面向校园局域网环境的轻量级即时通信与文件传输系统，采用客户端/服务器（C/S）架构，提供点对点文本聊天、群聊、文件传输和历史记录查看等核心功能，满足校园场景下高效、简洁的通信需求。

1.1.2 目标

开发一个功能完整、运行稳定的即时通信系统，具体要求如下：

- （1）采用 C/S 架构，服务端支持多客户端并发连接。
- （2）用户可通过账号密码登录系统，支持新用户注册。
- （3）支持在线用户列表查看、点对点文本聊天和群聊。
- （4）支持文件在线传输并显示传输进度。
- （5）聊天记录持久化存储，支持历史记录查看。
- （6）具备基本的异常处理能力（断线检测、重连、状态更新）。

1.1.3 适用范围

本系统面向校园局域网环境，服务端部署在局域网内的一台主机上，客户端可运行在同一局域网内的多台计算机上。系统主要用于课程设计教学演示和技术实践。

1.2 可行性研究

1.2.1 技术可行性

推荐技术选型：Python 3.10+ + Socket + tkinter + SQLite。所有依赖均为 Python 标准库，零额外安装，开发周期短，AI 辅助成熟度高。Python 的 socket 库提供了完整的 TCP Socket

编程接口，支持多线程并发处理；tkinter 作为 Python 标准 GUI 库，能够满足本系统对图形界面的需求；SQLite 作为嵌入式数据库，无需独立安装和配置，适合轻量级数据存储场景。以上技术在课程教学中均有涉及，小组成员具备相应的技术基础。

1.2.2 经济可行性

无需购买任何付费软件或服务，全部使用开源免费技术。硬件需求低，一台普通 PC 即可同时运行服务端和多个客户端进行测试。开发周期短（10 天），人力成本可控。

1.2.3 操作可行性

用户群体为在校学生，对即时通信软件的操作模式非常熟悉。系统功能对标日常使用的 IM 工具，操作逻辑直观。服务端启动后无需人工干预，客户端提供图形界面，操作简单。

1.2.4 结论

经技术、经济、操作三方面分析，本项目完全可行，采用 Python + Socket + tkinter + SQLite 技术方案进行开发。

1.3 功能需求

1.3.1 功能列表

系统功能需求分为高、中、低三个优先级，具体如下表所示：

编号	功能名称	功能描述	优先级
F01	用户注册与登录	用户通过用户名和密码注册账号并登录系统，密码采用 SHA256 哈希存储	高
F02	在线用户列表	登录后显示当前所有在线用户，用户上线/下线时列表实时更新	高
F03	点对点私聊	双击在线用户打开私聊窗口，支持文本消息的发送和接收，气泡式展示	高
F04	公共聊天室	提供公共聊天室（群聊）功能，消息广播给所有在线用户	高

F05	用户状态管理	服务端管理用户在线状态，用户登录/登出时广播状态变更通知	高
F06	文件传输	支持在线用户之间点对点文件传输，包含传输进度显示	中
F07	聊天历史记录	私聊和群聊消息持久化存储，支持查看历史聊天记录	中
F08	心跳保活与重连	客户端定期发送心跳包，断线后自动尝试重连	中

1.3.2 功能优先级说明

高优先级（F01-F05）：对应课程设计指导书基本要求，是系统必须实现的核心功能，需在 6 月 25 日前完成并集成运行。

中优先级（F06-F08）：对应扩展功能，在基本功能完成且时间允许的情况下实现，增强系统的实用性和完整性。

1.4 非功能需求

类别	需求描述
性能需求	服务端支持至少 10 个客户端同时在线，私聊消息转发延迟不超过 1 秒，文件传输支持 100MB 以内文件
可靠性需求	服务端 7×24 小时稳定运行，单个客户端异常断开不影响服务端和其他客户端正常运行
安全性需求	密码使用 SHA256 哈希存储，通信内容不加密（局域网环境），防止重复登录
可用性需求	客户端 GUI 界面美观友好，操作流程符合用户习惯，提供清晰的错误提示信息
可维护性需求	代码模块化设计，通信层与 UI 层分离，具备单元测试覆

1.5 用例分析

1.5.1 用户角色定义

系统仅包含一种用户角色——普通用户。所有用户具有相同的功能权限，均可使用注册、登录、私聊、群聊、文件传输、历史记录查询等功能。

1.5.2 用例图

系统包含 9 个核心用例：注册账号、用户登录、查看在线用户、点对点文本聊天、群聊/公共聊天室、查看聊天历史记录、发送文件、接收文件、退出登录。所有用例关联同一参与者「普通用户」。

用例关系说明：用户登录包含查看在线用户（登录成功自动获取在线用户列表）；点对点文本聊天扩展查看聊天历史记录（私聊时可查阅历史消息）；群聊/公共聊天室扩展查看聊天历史记录（群聊时可查阅历史消息）；发送文件包含查看在线用户（文件传输需选择在线用户作为接收方）。

1.5.3 核心用例描述

用例 UC02：用户登录。基本流程：用户启动客户端，输入用户名和密码，点击登录按钮，客户端连接服务端并发送 LOGIN 消息，服务端验证用户名和密码，验证通过后加入在线列表并广播 USER_ONLINE 通知，返回 SUCCESS 消息（含在线用户列表）给客户端，客户端显示主界面。异常流程：服务端未启动或 IP/端口错误时客户端显示连接失败提示；用户名不存在时服务端返回"用户名不存在"错误；密码错误时服务端返回"密码错误"错误。

用例 UC04：点对点文本聊天。基本流程：用户在在线用户列表中双击目标用户，打开私聊窗口，输入消息文本后点击发送，客户端封装 PRIVATE_MSG 消息发送至服务端，服务端查找目标用户连接并转发消息，同时写入数据库，接收方客户端显示消息。异常流程：接收方已离线时服务端返回"对方已离线"提示给发送方。

用例 UC07/UC08：文件传输。基本流程：发送方在私聊窗口中点击发送文件按钮，选择本地文件，客户端发送 FILE_REQUEST 消息经服务端转发至接收方，接收方弹出确认框，接受后发送 FILE_RESPONSE，发送方分块发送 FILE_DATA，接收方逐块接收并显示进度，传输完成后发送 FILE_COMPLETE。异常流程：传输过程中接收方断开时服务端通知发送

方传输中断；接收方拒绝接收时服务端通知发送方"对方拒绝接收"。

1.6 数据流图

1.6.1 顶层数据流图

顶层数据流图（上下文图）展示系统与外部实体之间的数据交互。用户 A 和用户 B 作为外部实体，与「校园即时通信与文件传输系统」进行双向数据流交互：用户向系统输入登录信息、消息文本、文件数据；系统向用户输出在线列表、消息、文件、历史记录。

1.6.2 第 1 层数据流图

系统内部划分为客户端和服务端两个子系统。客户端包含用户界面和客户端 Socket 模块，负责用户交互和网络通信。服务端包含服务端 Socket 模块（接收连接）、认证管理（登录验证）、在线用户管理（状态维护）和消息路由引擎（消息转发），消息路由引擎与聊天记录数据库交互实现持久化存储。

1.7 E-R 图与数据字典

1.7.1 E-R 图

系统包含三张核心数据表：USER（用户表）、PRIVATE_MESSAGE（私聊消息表）、GROUP_MESSAGE（群聊消息表）。USER 与 PRIVATE_MESSAGE 为一对多关系，一个用户可发送/接收多条私聊消息。USER 与 GROUP_MESSAGE 为一对多关系，一个用户可发送多条群聊消息。

1.7.2 数据字典

（1）用户表（USER）：

字段名	类型	长度	约束	说明
username	VARCHAR	32	PRIMARY KEY	用户名，唯一标识
password	VARCHAR	64	NOT NULL	密码（SHA256 哈希值）
nickname	VARCHAR	32		用户昵称
status	VARCHAR	16	DEFAULT 'offline'	在线状态： online/offline

(2) 私聊消息表 (PRIVATE_MESSAGE):

字段名	类型	约束	说明
id	INTEGER	PRIMARY KEY AUTOINCREMENT	消息唯一 ID
sender	VARCHAR(32)	NOT NULL, FK→user	消息发送者
receiver	VARCHAR(32)	NOT NULL, FK→user	消息接收者
content	TEXT	NOT NULL	消息内容 (文本)
timestamp	DATETIME	NOT NULL	消息发送时间
type	VARCHAR(16)	DEFAULT 'text'	消息类型: text/file

(3) 群聊消息表 (GROUP_MESSAGE):

字段名	类型	约束	说明
id	INTEGER	PRIMARY KEY AUTOINCREMENT	消息唯一 ID
sender	VARCHAR(32)	NOT NULL, FK→user	消息发送者
group_id	VARCHAR(32)	NOT NULL	群组标识
content	TEXT	NOT NULL	消息内容
timestamp	DATETIME	NOT NULL	消息发送时间

(4) 通信协议消息格式: 所有消息采用 JSON 格式, 通过 TCP Socket 传输, 以换行符 (\n) 作为消息分隔符。每条消息包含 type (消息类型)、from (发送者)、timestamp (时间戳) 等基础字段, 根据消息类型不同可包含 password、content、receiver、filename、filesize、data 等扩展字段。

1.8 验收标准

编号	验收项	验收标准	是否通过
A01	服务端启动	服务端启动后正常监听指定端口, 控制台输出启动信息	通过

A02	用户注册	新用户可通过客户端注册账号，重复用户名提示错误	通过
A03	用户登录	已注册用户可通过正确密码登录，错误密码提示失败	通过
A04	在线用户列表	登录成功后显示在线用户列表，新用户上线/下线时列表实时更新	通过
A05	点对点私聊	两个在线用户可互相发送文本消息，消息正确显示	通过
A06	群聊广播	群聊消息可被所有在线用户（除发送者）收到	通过
A07	文件传输	文件可在两个在线用户之间传输，接收方文件内容与发送方一致	通过
A08	历史记录	可查看私聊和群聊的历史消息记录	通过
A09	断线处理	客户端异常断开后，服务端在 30 秒内心跳超时检测到并更新状态	通过
A10	多客户端并发	至少 3 个客户端同时在线，消息不串扰、不丢失	通过

二、概要设计

2.1 系统架构设计

2.1.1 总体架构

本系统采用经典的客户端/服务器（C/S）两层架构。服务端是整个系统的核心，负责监听客户端连接、管理在线用户状态、路由和转发消息、持久化聊天记录。客户端负责提供用户交互界面，通过 TCP Socket 与服务端建立持久连接，发送用户操作请求并接收服务端推送的消息。

系统架构分为三个层次：表示层（客户端 GUI 界面，基于 tkinter 实现）、业务逻辑层（服务端消息路由引擎，基于多线程 TCP Server 实现）、数据持久层（SQLite 数据库，存储用户信息和聊天记录）。

2.1.2 架构图描述

客户端层		
客户端 A (tkinter GUI)	客户端 B (tkinter GUI)	客户端 C (tkinter GUI)
TCP Socket 通信（JSON 消息封装）		
服务端（多线程 TCP Server）		
认证管理模块	在线用户管理模块	消息路由引擎
聊天记录持久化模块		
SQLite 数据库		

2.2 模块划分

2.2.1 服务端模块

模块名称	文件	功能描述	负责人
通信协议定义	protocol.py	定义 JSON 消息格式、消息类型枚举、消息构建/解析函数	李坤灿
数据库模块	db.py	SQLite 数据库初始化、用户 CRUD、消息存储与查	李坤灿

		询	
服务端核心	server.py	TCP 监听、多线程并发、 用户认证、消息路由、在 线管理	李坤灿

2.2.2 客户端模块

模块名称	文件	功能描述	负责人
客户端核心	client.py	Socket 连接管理、消息发 送/接收、断线检测与重 连、回调机制	侯皓骞
公共工具	ui_utils.py	GUI 配色方案、头像工具 函数、通用 UI 组件	侯皓骞
登录界面	login_window.py	用户注册/登录 GUI、服务 器 IP/端口配置	侯皓骞
主界面	main_window.py	在线用户列表、导航栏、 状态栏、消息分发	侯皓骞
私聊窗口	chat_window.py	一对一聊天、气泡消息展 示、文件传输界面	侯皓骞
群聊窗口	group_chat_window.py	公共聊天室界面、群消息 广播显示	董鑫

2.2.3 辅助模块

模块名称	文件	功能描述	负责人
协议测试	test_protocol.py	通信协议模块单元测试	李坤灿
数据库测试	test_db.py	数据库模块单元测试	李坤灿
用户管理工具	manage_users.py	命令行用户账号管理工具	李坤灿
打包脚本	build_server.py/build_client.py	PyInstaller 打包脚本	李坤灿

2.3 技术选型

技术项	选型	选型理由
编程语言	Python 3.10+	语法简洁，标准库丰富，开发效率高，课程教学语言
GUI 框架	tkinter	Python 标准库，无需额外安装，零依赖，满足基本 GUI 需求
数据库	SQLite 3	嵌入式数据库，无需安装配置，适合轻量级数据存储，Python 标准库支持
通信协议	TCP Socket + JSON	TCP 保证可靠传输，JSON 格式通用可读，便于调试和扩展
打包工具	PyInstaller	可将 Python 程序打包为独立 EXE 文件，便于分发部署
版本控制	Git	支持分支协作开发，代码审查和版本回溯

三、详细设计

3.1 通信协议设计

3.1.1 消息格式定义

系统采用 JSON 作为应用层消息格式，所有消息通过 TCP Socket 传输。每条消息以换行符(\n)作为分隔符，接收端按行解析。所有消息采用统一的 JSON 结构，包含 type（消息类型）、from（发送者用户名）、timestamp（时间戳）基础字段，并根据消息类型扩展 receiver、content、password、filename、filesize、data、online_list、records 等字段。

3.1.2 消息类型定义

类型常量	值	方向	说明
LOGIN	LOGIN	C→S	用户登录请求
REGISTER	REGISTER	C→S	用户注册请求
LOGOUT	LOGOUT	C→S	用户登出通知
SUCCESS	SUCCESS	S→C	操作成功响应
ERROR	ERROR	S→C	错误消息
PRIVATE_MSG	PRIVATE_MSG	C→S→C	私聊消息
GROUP_MSG	GROUP_MSG	C→S→C	群聊消息
USER_ONLINE	USER_ONLINE	S→C	用户上线广播
USER_OFFLINE	USER_OFFLINE	S→C	用户下线广播
FILE_REQUEST	FILE_REQUEST	C→S→C	文件传输请求
FILE_RESPONSE	FILE_RESPONSE	C→S→C	文件接收响应
FILE_DATA	FILE_DATA	C→S→C	文件数据块
FILE_COMPLETE	FILE_COMPLETE	C→S→C	文件传输完成
HEARTBEAT	HEARTBEAT	C→S	心跳包
HISTORY_REQUEST	HISTORY_REQUEST	C→S	历史记录请求
HISTORY_RESPONSE	HISTORY_RESPONSE	S→C	历史记录响应
QUIT	QUIT	C→S	客户端主动退出

3.1.3 通信时序说明

① 用户登录时序：客户端发送 LOGIN 消息 → 服务端验证用户名密码 → 加入在线列表 → 广播 USER_ONLINE 给其他客户端 → 返回 SUCCESS（含在线用户列表）给请求客户端。

② 点对点私聊时序：发送方客户端发送 PRIVATE_MSG 消息 → 服务端查找接收方连接 → 写入数据库 → 转发 PRIVATE_MSG 给接收方客户端 → 接收方显示消息。

③ 文件传输时序：发送方发送 FILE_REQUEST → 服务端转发给接收方 → 接收方响应 FILE_RESPONSE（接受/拒绝）→ 服务端转发响应 → 发送方分块发送 FILE_DATA → 服务端逐块转发 → 接收方逐块接收并显示进度 → 传输完成后发送 FILE_COMPLETE。

3.2 数据库设计

3.2.1 E-R 图

系统包含三个核心实体：用户（USER）、私聊消息（PRIVATE_MESSAGE）、群聊消息（GROUP_MESSAGE）。USER 与 PRIVATE_MESSAGE 是一对多关系，一个用户可发送/接收多条私聊消息。USER 与 GROUP_MESSAGE 是一对多关系，一个用户可发送多条群聊消息。

3.2.2 数据表设计

用户表（user）包含字段：username（VARCHAR(32), PRIMARY KEY）、password（VARCHAR(64), NOT NULL, SHA256 哈希值）、nickname（VARCHAR(32)）、status（VARCHAR(16), DEFAULT offline）。

私聊消息表（private_message）包含字段：id（INTEGER, PRIMARY KEY AUTOINCREMENT）、sender（VARCHAR(32), FK→user）、receiver（VARCHAR(32), FK→user）、content（TEXT, NOT NULL）、timestamp（DATETIME, NOT NULL）、type（VARCHAR(16), DEFAULT text）。

群聊消息表（group_message）包含字段：id（INTEGER, PRIMARY KEY AUTOINCREMENT）、sender（VARCHAR(32), FK→user）、group_id（VARCHAR(32), NOT NULL）、content（TEXT, NOT NULL）、timestamp（DATETIME, NOT NULL）。

3.3 核心类设计

3.3.1 类图概述

类名	所属模块	职责	主要方法
ChatServer	server.py	服务端核心，监听端口、 接受连接	start(), __init__(host, port)
ClientManager	server.py	线程安全地管理在线客户 端连接	add(), remove(), get(), send_to(), broadcast()
MessageRouter	server.py	根据消息类型分发到对应 处理器	route(), handle_login(), handle_private_msg()等
ClientHandler	server.py	单客户端处理线程	run(), _reply(), _cleanup()
ChatClient	client.py	客户端网络通信核心	connect(), disconnect(), login(), send_private_msg(), send_file_request()等
LoginWindow	login_window.py	登录/注册界面	_setup_ui(), _do_login(), _do_register()
MainWindow	main_window.py	主界面	_update_user_list(), _on_private_msg()
ChatWindow	chat_window.py	私聊窗口	add_message(), _send_message(), _send_file()
GroupChatWindow	group_chat_window.py	群聊窗口	add_message(), _send_message()

3.3.2 核心类详细设计

ChatClient 类（客户端核心）：通过回调机制实现通信层与 UI 层的解耦。核心属性包括 `_socket`（TCP 连接）、`_host/_port`（服务器地址）、`_username`（当前用户）、`_running`（运行状态）、`_callbacks`（回调函数字典）、`_file_buffers`（文件传输缓冲区）。核心方法包括 `connect()` 建立 TCP 连接并启动监听线程和心跳线程、`login()` 发送登录请求、`send_private_msg()/send_group_msg()` 发送聊天消息、`send_file_request()/start_send_file()` 发送文件、`_listen_loop()` 持续监听服务端消息、`_dispatch()` 根据消息类型触发对应回调。

ChatServer 类（服务端核心）：包含 `ClientManager`（线程安全的客户端连接管理器）和

MessageRouter（消息路由引擎）。start()方法初始化数据库、创建 ServerSocket、进入 accept() 主循环，每接受一个新连接创建 ClientHandler 线程。MessageRouter 维护消息类型到处理函数的映射表，处理函数包括 handle_login()（用户认证）、handle_private_msg()（私聊转发+写入数据库）、handle_group_msg()（群聊广播+写入数据库）、handle_file_request/data/complete()（文件传输转发）、handle_history_request()（历史记录查询）等。

3.4 核心流程设计

3.4.1 用户登录流程

步骤 1：用户输入 IP、端口、用户名、密码。步骤 2：客户端连接服务端，失败则提示"连接失败"。步骤 3：发送 LOGIN 消息。步骤 4：服务端验证用户名和密码，用户名不存在或密码错误则返回 ERROR。步骤 5：验证通过后加入在线列表，广播 USER_ONLINE 给其他客户端。步骤 6：返回 SUCCESS（含在线用户列表）给客户端。步骤 7：客户端设置用户名，显示主界面。

3.4.2 消息转发流程

步骤 1：发送方输入消息并点击发送。步骤 2：客户端封装 PRIVATE_MSG 消息。步骤 3：发送至服务端。步骤 4：服务端根据 receiver 字段查找接收方连接。步骤 5：接收方在线则转发消息并写入数据库；接收方离线则返回 ERROR 提示。步骤 6：接收方客户端显示消息。

3.4.3 异常断线处理流程

步骤 1：客户端异常断开（网络中断/程序崩溃）。步骤 2：服务端心跳检测超时（30 秒无心跳）或 recv()返回空数据。步骤 3：服务端从在线列表移除该用户，更新数据库状态为 offline。步骤 4：广播 USER_OFFLINE 给所有在线客户端。步骤 5：各客户端更新在线用户列表，UI 实时刷新。步骤 6：服务端继续正常运行，异常隔离。

3.5 界面设计

3.5.1 登录界面

登录界面采用现代化设计风格，尺寸 420×520 像素，居中显示。顶部为蓝色 Header 区域，包含 IM 图标和"校园即时通信"标题；中部为白色卡片区域，包含可折叠的服务器设置（默认 127.0.0.1:8888）、用户名输入框和密码输入框；底部为蓝色登录按钮和绿色注册按钮。

支持回车键快捷登录。

3.5.2 主界面

主界面尺寸 860×600 像素，最小 700×450 像素。采用左右分栏布局：左侧为在线用户列表（带在线人数统计和头像），右侧为欢迎区/聊天区。顶部蓝色导航栏包含"公共聊天室"和"退出"按钮。底部状态栏显示连接状态和系统消息。支持双击在线用户打开私聊窗口。

3.5.3 私聊窗口

私聊窗口尺寸 550×600 像素，采用仿微信聊天气泡风格。顶部显示对方头像、用户名和在线状态。消息展示区使用 Canvas+Frame 实现滚动，自己的消息以蓝色气泡右对齐显示，对方消息以白色气泡左对齐显示（含头像），时间戳居中显示。底部输入区支持多行文本输入和回车发送。顶部工具栏包含发送文件按钮和历史记录按钮。

3.5.4 群聊窗口

群聊窗口尺寸 600×600 像素，布局与私聊窗口类似。区别在于标题显示"公共聊天室"，对方的每条消息显示发送者用户名和头像，所有在线用户均可看到彼此发送的消息。支持查看群聊历史记录。

四、系统实现

4.1 开发环境与工具

项目	说明
操作系统	Windows 11
编程语言	Python 3.12
IDE/编辑器	VS Code / Trac IDE
GUI 框架	tkinter (Python 标准库)
数据库	SQLite 3 (Python 标准库 sqlite3)
版本控制	Git
AI 辅助工具	Trac IDE (内置 AI 编程助手)
打包工具	PyInstaller

4.2 服务端实现

4.2.1 服务端启动流程

服务端启动时首先调用 `init_db()` 初始化 SQLite 数据库（创建 `user`、`private_message`、`group_message` 三张表），然后创建 `ServerSocket` 绑定指定 IP 和端口（默认 `0.0.0.0:8888`），进入主循环调用 `accept()` 阻塞等待客户端连接。每接受一个新连接，创建 `ClientHandler` 线程处理该客户端的后续通信。服务端使用 `SO_REUSEADDR` 选项允许端口复用，支持 `Ctrl+C` 优雅关闭。

4.2.2 多线程并发模型

每个客户端连接对应一个独立的 `ClientHandler` 线程（继承自 `threading.Thread`，设置为 `daemon` 模式）。`ClientHandler` 在 `run()` 方法中循环接收客户端消息，按换行符分割为完整 JSON 消息后调用 `MessageRouter.route()` 进行分发处理。若 `recv()` 返回空数据或抛出异常，则执行 `cleanup()` 清理：从 `ClientManager` 中移除连接、更新数据库状态为 `offline`、广播 `USER_OFFLINE` 通知。

4.2.3 消息路由引擎

`MessageRouter` 维护一个消息类型到处理函数的字典映射。处理函数包括：`handle_login()`

验证用户密码，检查重复登录，设置在线状态，广播上线通知，返回在线用户列表；
`handle_register()`检查用户名唯一性后插入数据库；`handle_private_msg()`保存消息到数据库并转发给接收方；`handle_group_msg()`保存消息到数据库并广播给所有在线用户（排除发送者）；
`handle_file_request/data/complete()`转发文件传输相关消息；`handle_history_request()`从数据库查询聊天记录并返回。

4.2.4 在线用户管理

`ClientManager` 类使用 `threading.Lock` 保证线程安全，内部维护 `{username: socket}` 字典。提供 `add()`、`remove()`、`get()`、`get_all_usernames()`等基本操作，以及 `send_to()`向指定用户发送消息、`broadcast()`向所有用户广播消息（支持排除指定用户）、`clean_dead_connections()`清理已断开的死连接。

4.2.5 数据库模块实现

数据库模块（`db.py`）封装了所有 SQLite 操作。`init_db()`在首次运行时自动创建三张数据表。`register_user()`使用 SHA256 哈希存储密码，通过捕获 `IntegrityError` 处理用户名重复。`verify_user()`先查询用户是否存在，再比对密码哈希值。消息存储函数（`save_private_msg`、`save_group_msg`）在服务端转发消息时调用，记录完整的时间戳。历史查询函数（`get_private_history`、`get_group_history`）按时间升序返回最近 100 条记录。

4.3 客户端实现

4.3.1 客户端核心框架

`ChatClient` 类实现了通信层与 UI 层的完全解耦。客户端通过 `set_callback()`注册回调函数，所有来自服务端的消息通过 `_dispatch()`方法根据消息类型触发对应的回调事件。支持的 16 种回调事件包括 `on_connected`、`on_disconnected`、`on_login_result`、`on_register_result`、`on_online_list`、`on_user_online`、`on_user_offline`、`on_private_msg`、`on_group_msg`、`on_file_request`、`on_file_response`、`on_file_progress`、`on_file_complete`、`on_history`、`on_error`、`on_progress`。

客户端启动后自动创建两个守护线程：`_listen_loop()`持续监听服务端消息并按行解析 JSON 分发，`_heartbeat_loop()`每 30 秒发送一次心跳包保持连接活跃。`connect()`方法设置 5 秒超时连接，成功后启动监听线程和心跳线程。`disconnect()`方法发送 QUIT 消息后关闭连接。

4.3.2 登录/注册界面实现

`LoginWindow` 类实现了美观的登录界面。顶部蓝色 Header 展示系统名称，中部白色卡

片包含可折叠的服务器设置（IP 地址和端口号，默认 127.0.0.1:8888）、用户名和密码输入框。登录和注册按钮分别对应蓝色和绿色配色，支持 hover 变色效果。通过 ChatClient 的回调机制处理登录/注册结果：登录成功后隐藏登录窗口并打开 MainWindow 主界面，登录失败显示错误提示。注册成功后提示用户点击登录按钮登录。

4.3.3 主界面实现

MainWindow 类实现了主界面框架。顶部蓝色导航栏包含"公共聊天室"和"退出"按钮。左侧边栏显示在线用户列表（使用 tkinter.Listbox），带在线人数统计徽章。右侧默认显示欢迎区（含用户头像和提示文字）。通过注册 ChatClient 的多个回调实现实时消息处理：on_user_online/offline 更新在线列表和状态栏，on_private_msg 自动打开或激活私聊窗口，on_group_msg 自动打开或激活群聊窗口，on_file_request/response/complete 处理文件传输通知。断线后自动尝试重连。

4.3.4 私聊窗口实现

ChatWindow 类实现了气泡式私聊界面。消息展示区使用 Canvas+Frame 实现，支持鼠标滚轮滚动。消息按发送者区分显示样式：自己的消息以蓝色气泡右对齐，对方的消息以白色气泡左对齐（含彩色头像圆圈）。系统消息和进度消息以灰色小字居中显示。输入区支持多行文本，Enter 键发送。文件传输功能：发送方通过文件选择对话框选择文件后发送 FILE_REQUEST，接收方弹出确认框，接受后发送方分块发送（每块 64KB，Base64 编码），接收方逐块接收并显示进度百分比，完成后保存到 received_files/目录。支持查看双方聊天记录。

4.3.5 群聊窗口实现

GroupChatWindow 类实现了公共聊天室界面。布局与私聊窗口类似，但标题显示"公共聊天室"，群聊消息显示发送者用户名和彩色头像。群聊消息通过 GROUP_MSG 类型发送，服务端广播给所有在线用户（排除发送者）。支持查看群聊历史记录。

4.4 关键功能实现说明

4.4.1 心跳保活与断线重连

客户端 ChatClient 在连接成功后启动心跳线程，每 30 秒向服务端发送 HEARTBEAT 消息。服务端 MessageRouter.handle_heartbeat()收到心跳后返回 None（不做任何响应），通过 TCP 层面的 recv()检测连接是否存活。若 recv()返回空数据或抛出异常，则认为客户端已断

开，执行清理流程。客户端监听线程检测到断开后触发 `on_disconnected` 回调，主界面在 3 秒后自动尝试重新连接。

4.4.2 文件传输实现

文件传输采用"请求-响应-分块传输-完成确认"四阶段协议。发送方先发送 `FILE_REQUEST` (含文件名、文件大小)，接收方弹出确认框后发送 `FILE_RESPONSE` (含 `accept` 字段)。若接收方接受，发送方以 64KB 为块大小读取文件，每块进行 Base64 编码后封装为 `FILE_DATA` 消息发送。接收方逐块接收并累积到 `bytearray` 中，计算接收进度百分比并通过 `on_file_progress` 回调更新UI。全部数据接收完成后，发送方发送 `FILE_COMPLETE` 消息，接收方将 `bytearray` 写入 `received_files/`目录。

4.4.3 密码安全存储

用户密码不存储明文，而是使用 SHA256 哈希算法进行单向加密。注册时将密码哈希值存入数据库，登录时将用户输入的密码计算哈希后与数据库中的哈希值比对。即使数据库文件泄露，也无法直接获取用户原始密码。

4.5 实际实现与设计调整

在系统实现过程中，根据实际开发情况对前期设计进行了以下调整和修订：

(1) 界面设计调整：原设计计划采用仿微信三栏布局（左侧导航栏+中间联系人列表+右侧聊天区），实际实现简化为两栏布局（左侧在线用户列表+右侧欢迎区/聊天区），顶部导航栏替代了左侧导航栏的功能，简化了界面复杂度，提高了开发效率。

(2) 配色方案调整：原设计采用微信绿色主色调，实际实现改为蓝色主色调（#2AABEE），视觉效果更加清新明快，符合校园应用场景。

(3) 文件传输编码：设计阶段计划使用二进制协议传输文件，实际实现改用 Base64 编码将文件数据嵌入 JSON 消息中传输。虽然增加了约 33%的数据量，但简化了协议实现，避免了二进制与文本混合传输的复杂性。

(4) 服务端架构简化：原设计计划使用独立的 `Database` 类和 `ConnectionManager` 类，实际实现将数据库操作封装为模块级函数（`db.py`），连接管理合并到 `ClientManager` 类中，减少了类的数量，代码更加紧凑。

(5) 群聊标识：设计阶段计划使用 `"__group__"` 作为群聊标识，实际实现使用 `"PUBLIC_ROOM"` 作为 `GROUP_ID` 常量，语义更明确。

(6) 消息类型调整：原设计中的 HISTORY 和 HISTORY_RESP 消息类型，实际实现改为 HISTORY_REQUEST 和 HISTORY_RESPONSE，命名更加规范一致。

五、系统测试

5.1 测试策略与环境

5.1.1 测试策略

本系统测试采用自底向上的策略，分为三个层次：单元测试（验证协议模块和数据库模块的正确性）、集成测试（验证客户端与服务端的协议对接和消息收发）、系统测试（验证完整的端到端功能流程）。测试覆盖正常功能、异常情况和并发场景，确保系统满足需求分析中定义的验收标准。

5.1.2 测试环境

配置项	参数
操作系统	Windows 11
Python 版本	Python 3.12
测试客户端数量	3 个
服务端地址	127.0.0.1:8888
测试数据库	test_chat.db（独立于生产数据库）
测试工具	Python unittest 框架

5.2 功能测试

5.2.1 单元测试

使用 Python unittest 框架对协议模块（test_protocol.py）和数据库模块（test_db.py）进行单元测试。

协议模块测试用例覆盖：

测试类	测试用例	测试内容	结果
TestMsgType	test_enum_values	验证消息类型枚举值正确性	通过
TestMsgType	test_group_id	验证 GROUP_ID 常量值	通过
TestBuildMessage	test_basic_message	验证消息构建包含必要字段	通过

TestParseMessage	test_valid_json	验证有效 JSON 解析	通过
TestParseMessage	test_bytes_input	验证 bytes 类型输入解析	通过
TestParseMessage	test_invalid_json	验证无效 JSON 返回 None	通过
TestGetField	test_get_field	验证字段提取和默认值	通过
TestConvenienceFunctions	8 个测试方法	验证所有便捷构建函数	通过

数据库模块测试用例覆盖：

测试类	测试用例	测试内容	结果
TestDB	test_sha256	验证 SHA256 哈希一致性	通过
TestDB	test_register_user	验证用户注册和重复注册 检测	通过
TestDB	test_verify_user	验证登录验证（正确密码、错误密码、不存在用户）	通过
TestDB	test_online_status	验证在线状态更新和查询	通过
TestDB	test_save_and_get_private_msg	验证私聊消息存储和双向 查询	通过
TestDB	test_save_and_get_group_msg	验证群聊消息存储和查询	通过

5.2.2 系统功能测试

系统功能测试覆盖了所有核心功能，测试过程如下：

测试编号	测试项	测试步骤	预期结果	实际结果
FT01	服务端启动	运行 <code>python -m src.core.server</code>	控制台输出监听信息，数据库初始化完成	通过
FT02	用户注册	客户端输入新用户名和密码，点击注册	提示"注册成功"，可继续登录	通过
FT03	重复注册	使用已存在的用户名注册	提示"用户名已存在"	通过

FT04	用户登录	输入正确的用户名和密码，点击登录	进入主界面，显示在线用户列表	通过
FT05	错误密码登录	输入正确的用户名但错误密码	提示"密码错误"	通过
FT06	不存在用户登录	输入不存在的用户名	提示"用户名不存在"	通过
FT07	在线用户列表	登录成功后查看在线用户列表	显示当前所有在线用户，含在线人数	通过
FT08	用户上线通知	新用户登录	其他在线客户端状态栏显示"XXX 上线了"	通过
FT09	用户下线通知	某用户退出	其他在线客户端状态栏显示"XXX 下线了"	通过
FT10	点对点私聊	用户 A 双击用户 B 发送消息	用户 B 的私聊窗口显示消息，气泡样式正确	通过
FT11	群聊消息	用户在公共聊天室发送消息	所有在线用户（除发送者）收到消息	通过
FT12	文件传输	用户 A 向用户 B 发送文件	用户 B 收到确认框，接受后文件正确传输并保存	通过
FT13	私聊历史记录	在私聊窗口点击历史记录	显示双方历史聊天记录	通过
FT14	群聊历史记录	在群聊窗口点击历史记录	显示群聊历史消息	通过
FT15	退出登录	点击退出按钮	返回登录界面，其他用户收到下线通知	通过

5.3 异常测试

测试编号	测试项	测试步骤	预期结果	实际结果
ET01	服务端未启动	客户端在服务端未启动时尝试连接	提示"连接失败"	通过
ET02	客户端断线	客户端强制关闭进程	服务端检测到断开，广播下线通知	通过
ET03	服务端重启	服务端重启后客户端重连	客户端自动重连并恢复在线状态	通过
ET04	空消息发送	不输入内容点击发送	消息不发送（前端拦截）	通过
ET05	重复登录	同一账号在另一个客户端登录	提示"该账号已在其他设备登录"	通过
ET06	向离线用户发消息	向已离线的用户发送私聊消息	提示"用户已离线"	通过
ET07	向离线用户发文件	向已离线的用户发送文件	提示"对方已离线，无法发送文件"	通过
ET08	传输中接收方断开	文件传输过程中接收方断开	服务端通知发送方传输中断	通过
ET09	拒绝文件接收	接收方拒绝文件传输请求	发送方收到"对方拒绝接收"提示	通过
ET10	非法消息格式	向服务端发送非JSON 数据	服务端返回"消息格式错误"	通过

5.4 并发测试

并发测试使用 3 个客户端同时在线，执行以下场景：

场景 1：3 个客户端同时登录，验证在线列表正确显示 3 个用户，每个客户端均能看到其他 2 个在线用户。结果：通过。

场景 2：用户 A 向用户 B 发送私聊消息，同时用户 B 向用户 C 发送私聊消息，验证消息不串扰。结果：通过，消息正确到达各自目标。

场景 3：3 个用户同时在公共聊天室发送消息，验证所有用户都能收到其他人的消息，

且不重复显示。结果：通过，消息广播正确，每个用户发送的消息不显示给自己。

场景 4：用户 A 向用户 B 发送文件的同时，用户 B 向用户 C 发送文件，验证两个传输互不干扰。结果：通过，两个文件传输同时进行，互不影响。

场景 5：一个客户端异常断开，验证其他客户端在线列表正确更新，服务端不崩溃。结果：通过，剩余客户端列表实时更新，服务端继续正常运行。

5.5 测试结果汇总

测试类别	测试用例数	通过数	失败数	通过率
单元测试（协议模块）	14	14	0	100%
单元测试（数据库模块）	6	6	0	100%
功能测试	15	15	0	100%
异常测试	10	10	0	100%
并发测试	5	5	0	100%

所有测试用例均通过，系统功能完整、运行稳定，满足需求分析中定义的全部验收标准。系统在 3 个客户端并发场景下表现良好，消息收发准确无误，文件传输完整可靠，异常处理机制有效。

核心源代码

以下展示系统核心模块的关键代码片段。完整源代码请参见项目 `src/` 目录。

1. 通信协议定义（`protocol.py` 核心部分）

```
class MsgType(str, Enum):
    """消息类型枚举"""
    REGISTER = 'REGISTER'      LOGIN = 'LOGIN'
    LOGOUT = 'LOGOUT'          PRIVATE_MSG = 'PRIVATE_MSG'
    GROUP_MSG = 'GROUP_MSG'    FILE_REQUEST = 'FILE_REQUEST'
    FILE_RESPONSE = 'FILE_RESPONSE' FILE_DATA = 'FILE_DATA'
    FILE_COMPLETE = 'FILE_COMPLETE' HEARTBEAT = 'HEARTBEAT'
    HISTORY_REQUEST = 'HISTORY_REQUEST'
    HISTORY_RESPONSE = 'HISTORY_RESPONSE'
    SUCCESS = 'SUCCESS'        ERROR = 'ERROR'
    QUIT = 'QUIT'              USER_ONLINE = 'USER_ONLINE'
    USER_OFFLINE = 'USER_OFFLINE'

def build_message(msg_type, sender, **kwargs):
    """构建标准JSON消息"""
    msg = {'type': msg_type, 'from': sender,
          'timestamp': time.strftime('%Y-%m-%d %H:%M:%S')}
    msg.update(kwargs)
    return json.dumps(msg, ensure_ascii=False)

def parse_message(raw_data):
    """解析收到的原始数据为Python dict"""
    if isinstance(raw_data, bytes):
        raw_data = raw_data.decode('utf-8')
    try:
        return json.loads(raw_data)
    except (json.JSONDecodeError, TypeError):
        return None
```

2. 服务端消息路由（`server.py` 核心部分）

```
class MessageRouter:
    def __init__(self, manager: ClientManager):
        self.manager = manager
        self._handlers = {
            MsgType.REGISTER: self.handle_register,
            MsgType.LOGIN: self.handle_login,
            MsgType.PRIVATE_MSG: self.handle_private_msg,
```

```

        MsgType.GROUP_MSG: self.handle_group_msg,
        MsgType.FILE_REQUEST: self.handle_file_request,
        MsgType.FILE_RESPONSE: self.handle_file_response,
        MsgType.FILE_DATA: self.handle_file_data,
        MsgType.FILE_COMPLETE: self.handle_file_complete,
        MsgType.HISTORY_REQUEST: self.handle_history_request,
        MsgType.HEARTBEAT: self.handle_heartbeat,
        MsgType.QUIT: self.handle_quit,
    }

def handle_login(self, sender, msg):
    username = get_field(msg, 'from')
    password = get_field(msg, 'password')
    success, reason = verify_user(username, password)
    if not success:
        return make_error('server', reason)
    if self.manager.get(username) is not None:
        return make_error('server', '该账号已在其他设备登录')
    set_user_online(username)
    self.manager.add(username, sender)
    online_broadcast = build_message(MsgType.USER_ONLINE, username)
    self.manager.broadcast(online_broadcast, exclude=username)
    online_list = self.manager.get_all_usernames()
    return build_message(MsgType.SUCCESS, 'server',
                        message=reason, online_list=online_list)

```

3. 客户端消息分发（**client.py** 核心部分）

```

def _dispatch(self, msg):
    """根据消息类型触发对应的回调"""
    msg_type = get_field(msg, 'type')
    if msg_type == MsgType.SUCCESS:
        online_list = get_field(msg, 'online_list', [])
        if online_list:
            self._fire('on_login_result', True, get_field(msg, 'message'))
            self._fire('on_online_list', online_list)
        else:
            self._fire('on_register_result', True, get_field(msg, 'message'))
    elif msg_type == MsgType.PRIVATE_MSG:
        self._fire('on_private_msg', get_field(msg, 'from'),
                  get_field(msg, 'content', ''), get_field(msg, 'timestamp'))
    elif msg_type == MsgType.GROUP_MSG:
        self._fire('on_group_msg', get_field(msg, 'from'),
                  get_field(msg, 'content', ''), get_field(msg, 'timestamp'))
    elif msg_type == MsgType.FILE_DATA:

```

```
filename = get_field(msg, 'filename', "")
total = get_field(msg, 'total', 0)
data_b64 = get_field(msg, 'data', "")
if filename not in self._file_buffers:
    self._file_buffers[filename] = {'data': bytearray(), 'total': total}
buf = self._file_buffers[filename]
chunk = base64.b64decode(data_b64)
buf['data'].extend(chunk)
buf['received'] = buf.get('received', 0) + len(chunk)
percent = int(buf['received'] / total * 100) if total > 0 else 0
self._fire('on_file_progress', filename, percent)
```

AI 辅助开发说明

1. 使用的 AI 工具

本小组在整个项目开发过程中，使用了以下 AI 工具进行辅助：

工具名称	版本/平台	使用成员	主要用途
Trac IDE	内置 AI 编程助手	李坤灿、侯皓骞、董鑫	代码生成、代码审查、调试辅助、文档撰写

2. AI 参与环节

AI 工具在项目开发的各个阶段均有参与，具体如下表所示：

开发阶段	AI 辅助内容	具体使用方式	输出成果
需求分析	需求文档框架生成、用例描述撰写、UML 图生成	向 AI 描述系统功能需求，AI 生成需求文档初稿和 Mermaid 格式的用例图、活动图、数据流图、E-R 图	需求分析文档初稿
系统设计	架构方案对比、通信协议设计、数据库设计、类图/时序图生成	向 AI 提问技术选型建议，提供需求描述让 AI 生成协议草案和数据库表结构	系统设计文档初稿
编码实现	代码框架生成、函数实现、bug 修复	向 AI 描述函数功能需求，AI 生成代码框架或具体实现；遇到错误时向 AI 描述错误信息，AI 辅助定位和修复	各模块源代码
代码审查	代码质量检查、潜在问题识别	将代码提交给 AI 审查，AI 检查代码规范、异常处理、线程安全等问题	审查意见和修改建议
系统测试	测试用例设计、测试代码	向 AI 描述测试需求，AI	测试代码

	生成	生成单元测试用例框架	
文档撰写	文档框架生成、内容填充、格式排版	向 AI 提供项目信息和文档要求，AI 生成各文档的初稿	最终项目文档初稿

3. AI 辅助的具体示例

3.1 示例 1：通信协议设计

提示词："请为校园即时通信系统设计 JSON 通信协议，需要支持登录、注册、私聊、群聊、文件传输、心跳检测等功能。请给出消息类型枚举和每种消息的 JSON 格式示例。"

AI 生成结果：AI 生成了完整的消息类型枚举（16 种消息类型）和对应的 JSON 格式示例，包括基础字段（type、from、timestamp）和扩展字段（password、content、receiver、filename、filesize、data 等）。

人工修改：小组成员对 AI 生成的协议进行了评审，将消息类型名从简写改为完整单词（如 MSG→PRIVATE_MSG），统一了命名规范，增加了群聊标识 GROUP_ID 常量，确认了消息分隔符为换行符。

3.2 示例 2：数据库设计

提示词："校园即时通信系统有用户表（username, password, nickname, status）、私聊消息表（id, sender, receiver, content, timestamp, type）、群聊消息表（id, sender, group_id, content, timestamp）。请生成 SQLite 建表 SQL 语句。"

AI 生成结果：AI 生成了三张表的 CREATE TABLE 语句，包含主键约束、外键约束、默认值设置。

人工修改：将 password 字段长度从 32 调整为 64（SHA256 哈希值长度为 64 字符），增加了 AUTOINCREMENT 和 NOT NULL 约束，将外键约束改为注释形式（SQLite 默认不强制外键）。

3.3 示例 3：GUI 界面代码生成

提示词："请用 Python tkinter 实现一个现代化的登录界面，包含用户名输入框、密码输入框、登录和注册按钮，使用蓝色主题配色。"

AI 生成结果：AI 生成了完整的 LoginWindow 类，包含界面布局、事件绑定、回调注册

等。

人工修改：调整了配色方案（从绿色改为蓝色），增加了可折叠的服务器设置区域，优化了按钮 `hover` 效果，添加了回车键快捷登录功能，增加了窗口居中显示。

3.4 示例 4: bug 修复辅助

提示词："服务端在客户端异常断开时，其他客户端没有收到 `USER_OFFLINE` 通知，请分析可能的原因。"

AI 分析结果：AI 指出 `ClientHandler` 的 `cleanup()` 方法中可能没有正确调用广播逻辑，建议检查 `finally` 块中的清理代码。

人工修复：确认问题后，在 `ClientHandler.cleanup()` 方法中增加了 `broadcast()` 调用，确保用户断开时广播 `USER_OFFLINE` 通知给所有在线客户端。

4. AI 使用原则与规范

本小组在 AI 使用过程中严格遵循以下原则：

- (1) 小步生成，小步验证：每次只让 AI 生成一个模块或一个功能，生成后立即验证正确性，发现问题及时修正。
- (2) 人工审查把关：所有 AI 生成的代码和设计必须经过人工审查，确保逻辑正确、符合项目规范。AI 生成的代码不得直接提交，必须经过理解和修改。
- (3) 提示词工程：使用结构化的提示词，明确输入输出格式，提高 AI 生成质量。
- (4) 迭代优化：AI 生成初稿后，人工提出修改意见，让 AI 进行迭代优化，或直接手动修改。
- (5) 记录使用过程：每位成员在个人开发日志中记录 AI 辅助的内容和效果，每条 AI 使用记录包含：使用的 AI 工具、参与环节、具体内容、人工修改情况。

5. 人工修改与验证情况

以下为各模块中 AI 生成内容经人工修改或验证的主要情况：

模块	AI 生成内容	人工修改/验证内容
通信协议	消息类型枚举和 JSON 格式	统一命名规范，增加 <code>GROUP_ID</code> 常量，确认消息分隔符，调整消息类型名称

数据库模块	建表 SQL 语句	调整字段长度，增加约束，优化索引
-------	-----------	------------------

		设计
服务端核心	多线程框架和消息路由	调整线程模型，增加异常处理，优化清理逻辑，修复广播通知缺失
客户端核心	Socket 连接和回调机制	增加断线重连逻辑，优化心跳间隔，完善文件传输缓冲区管理
登录界面	界面布局和事件处理	调整配色方案，增加服务器设置折叠，优化交互细节
主界面	在线用户列表和导航栏	简化布局（三栏改两栏），增加在线人数统计，优化重连逻辑
私聊窗口	气泡消息和文件传输	调整气泡样式，优化文件传输进度显示，增加历史记录加载
群聊窗口	群聊消息展示	增加发送者头像和用户名显示，优化消息广播逻辑
测试代码	单元测试用例	增加边界条件测试，完善异常场景覆盖
文档	各文档初稿	补充实际实现细节，调整章节结构，统一格式和术语

6. AI 辅助效果评估

评估维度	传统方式预估	AI 辅助实际	效率提升
需求分析文档	4-6 小时	1-2 小时	约 70%
系统设计文档	4-6 小时	1-2 小时	约 70%
通信协议设计	1-2 小时	20 分钟	约 80%
数据库设计	1-2 小时	30 分钟	约 70%
代码框架生成	3-4 小时	1 小时	约 70%
单元测试编写	2-3 小时	30 分钟	约 80%
最终文档撰写	6-8 小时	2-3 小时	约 65%

AI 辅助开发显著提升了开发效率，尤其是在文档撰写、协议设计和测试编写方面效果明显。但 AI 生成的代码并非完全可靠，仍需人工审查和修改，特别是在异常处理、线程安全和业务逻辑正确性方面需要重点关注。总体而言，AI 作为辅助工具，有效加速了开发进程，但核心决策和质量把控仍由小组成员完成。